

Kontext pro kódovací agenty

Kontext pro AI vývoj se dá rozdělit do tří hlavních částí:

1. **Projekt**
2. **Úkol**
3. **Agent**

Každá z těchto částí se chová trochu jinak a je dobré o nich přemýšlet separátně.

1. Projekt

První část kontextu je samotný projekt, typicky Git repozitář.

V rámci repozitáře mají různé soubory různou důležitost.

Typicky jsou velmi důležité například:

- `README.md`
- `AGENTS.md`
- `CLAUDE.md`
- konfigurace projektu
- struktura adresářů
- běžná místa daného frameworku

Pokud projekt používá například Next.js, agent už zná určité konvence a očekává, že některé věci budou na obvyklých místech.

1. Projekt — struktura

Obecně platí:

Čím výše v adresářové struktuře se informace nachází, tím globálnější význam typicky má.

Například:

```
/
├── README.md
├── AGENTS.md
├── package.json
├── src/
│   ├── app/
│   │   ├── ...
│   │   └── feature/
│   │       └── ...
│   └── ...
```

`README.md` nebo `AGENTS.md` popisují většinou celý projekt. Naopak informace hluboko uvnitř konkrétního adresáře bývají relevantní hlavně pro danou část projektu.

Git historie

Speciální součástí projektového kontextu může být Git historie.

Někdy chceme, aby agent mohl používat:

```
git log  
git blame  
git diff
```

a pochopil například:

- proč nějaký kód vznikl,
- jak se projekt vyvíjel,
- kdo a proč něco změnil.

Jindy naopak chceme, aby agent pracoval pouze s aktuálním stavem `workdir` a minulost ignoroval. Tohle je dobré rozhodovat explicitně.

2. Úkol

Druhou částí kontextu je úkol, na kterém se právě pracuje.

Může mít podobu například:

- PRD,
- specifikace,
- jednoho promptu,
- promptu pro Claude Code nebo Codex,
- celé konverzace s agentem.

Příklad úkolu

Projekt:
Promptbook

Úkol:
Přidej možnost propojit projekt s workshopem.

Požadavky:

- jeden projekt může být propojen s více workshopy
- propojení musí být editovatelné
- přidej testy

Projekt vs. Úkol

Projektový kontext odpovídá na otázku:

■ Jak tento systém funguje?

Úkol odpovídá na otázku:

■ Co děláme právě teď?

3. Agent

Třetí částí kontextu je to, **kdo úkol provádí**.

Agent má několik vrstev.

Model

Nejnižší vrstva je samotný model.

Například:

```
GPT
Claude
Gemini
Qwen
lokální model
fine-tuned model
```

Model určuje základní schopnosti, způsob uvažování, znalosti a limity.

Harness

Nad modelem je harness - prostředí, které model používá pro práci s kódem.

Například:

```
Codex  
Claude Code  
OpenCode  
Qwen Code  
vlastní coding agent
```

Harness typicky řeší například čtení souborů, editaci souborů, spouštění příkazů, správu konverzace a předávání kontextu modelu.

Stejný model se může v různých harnesses chovat výrazně jinak.

Skills, instrukce a nástroje

Další vrstvu tvoří schopnosti a instrukce přidané nad samotný harness.

Například:

```
Custom instructions  
Skills  
Tools  
MCP servery  
API  
Browser  
GitHub  
Databáze  
Interní firemní nástroje
```

Ty určují, k čemu má agent přístup a jakým způsobem má pracovat.

Meta orchestrátor

Nad jednotlivými agenty může být ještě další vrstva – meta orchestrátor.

Ten například rozhoduje:

- který agent má úkol řešit,
- jaký model použít,
- jaký prompt vytvořit,
- jaké nástroje připojit,
- kdy agenta spustit znovu,
- jak vyhodnotit výsledek.

Příkladem je Promptbook Coder: <https://coder.ptbk.io/>

Vrstvy agenta

Meta orchestrátor



Skills / Tools / Instructions



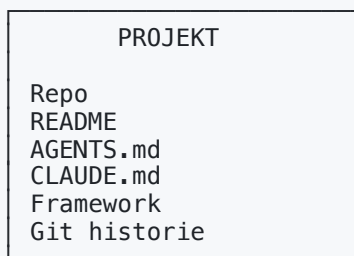
Harness



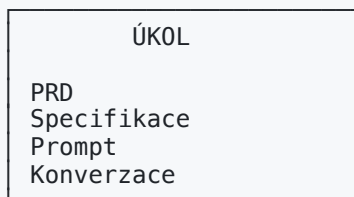
Model

Celkový model

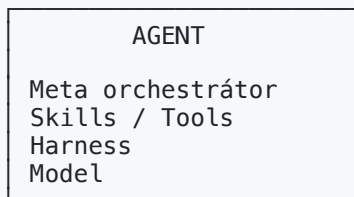
Celý kontext AI vývoje můžeme zjednodušit takto:



+



+



Tyto tři části dohromady tvoří kontext, ve kterém coding agent pracuje.

Kde jednotlivé části žijí

Projekt

Projektový kontext je typicky:

per projekt

Sdílí ho všichni vývojáři pracující na daném repozitáři.

Úkol

Kontext úkolu je typicky:

per vývojář × per úkol

Každý vývojář může ve stejném projektu řešit jinou věc.

Agent

Konfigurace agenta může být například:

per vývojář

Typický příklad: každý vývojář má vlastní Claude Code nebo OpenAI Codex.

Nebo:

per projekt / per firma

Firma může definovat jednotný způsob, jakým agenti pracují, například přes vlastní orchestrátor typu Promptbook Coder.

Shrnutí

Při práci s coding agenty je užitečné oddělovat tři otázky:

1. V jakém projektu pracuji?
2. Co právě dělám?
3. Kdo to dělá a s jakými schopnostmi?

Tedy

PROJECT
+
TASK
+
AGENT
=
CONTEXT